# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

# Case study

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN**
**COURSE CODE: CSA1110**

---

**COURSE OUTCOME COVERED**

**CO3:** Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models**.** *(BL4)*

Assessment Tool Weightage: 40%

---

***Code of Conduct:***

*I **V V NAVEEN KUMAR/192321018** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***
***naveenkumarvv***

# ANSWERS TO SELECTED CASE STUDIES (Question 1, 2, 3 & 4)

## 1. Online Hotel Reservation System

*A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions. Question: Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.*

**(10 Marks)**

**ANSWER**

### 1.1 System Understanding

The Online Hotel Reservation System has three principal actors – the Customer, who searches hotels, books rooms, pays, and posts reviews; the Hotel Manager, who updates room availability and pricing; and the Administrator, who manages user accounts and transactions. Because each actor performs a distinct set of operations on a common pool of data (hotels, rooms, bookings), the system is a natural fit for object-oriented analysis and design.

### 1.2 Application of Object-Oriented Principles

- Encapsulation: The Payment class hides sensitive details such as card number and CVV inside private attributes, exposing only a processPayment() method. Similarly, the Room class encapsulates its price-calculation logic (taxes, seasonal discounts) behind a getFinalPrice() method, so other classes never manipulate the raw pricing data directly.
- Inheritance: A common base class User holds shared attributes (userId, name, email, password) and behaviour (login(), logout()). Customer, HotelManager, and Administrator all extend User and add role-specific behaviour – e.g., Customer adds bookRoom() and postReview(), HotelManager adds updateAvailability() and setPricing(), and Administrator adds manageUsers() and viewTransactionLogs().
- Polymorphism: A method such as generateReport() is defined in the User hierarchy but overridden differently by HotelManager (produces an occupancy/revenue report) and Administrator (produces a user-activity/audit report). Likewise, notify() can be overridden to send an SMS, email, or in-app alert depending on the subclass that implements it, allowing the same method call to behave differently across the hierarchy.

### 1.3 Recommended SDLC Model

The Agile (Scrum) model is recommended for this system. Hotel booking platforms have core requirements (search, booking, payment) that are reasonably clear at the outset, but features such as review handling, dynamic pricing, and third-party payment gateway integration tend to evolve as real users and hotel partners give feedback. Agile allows the core booking module to be delivered early as a working increment, with reviews, pricing rules, and reporting features

refined in subsequent sprints based on stakeholder feedback. This reduces the risk of building unwanted features and lets the travel company respond quickly to changing business needs, which a rigid, sequential model such as Waterfall would not support well.

---

# 2. University Management System

*A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system. Question: Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.*

**(10 Marks)**

**ANSWER**

## 2.1 System Understanding

The University Management System automates four major processes – admissions, course registration, examination management, and result processing – each involving a different set of users: Student, Faculty, Examination Staff, and Administrator. A structured object-oriented approach begins by identifying the real-world entities in these processes as classes, and the interactions between them as relationships.

## 2.2 Key Classes and Relationships

- Person (abstract base class): personId, name, email, contactNumber – parent of Student, Faculty, ExaminationStaff, and Administrator.
- Student: registrationNo, department, semester | register(), viewResult().
- Faculty: facultyId, department, designation | assignGrades(), conductClass().
- Course: courseId, courseName, credits, department | addCourse(), updateSyllabus().
- Registration: registrationId, registrationDate | enrollStudent() – links a Student to one or more Courses (many-to-many).
- Examination: examId, examDate, courseId | scheduleExam() – conducted by ExaminationStaff for a Course.
- Result: resultId, marks, grade | generateResult() – associated with one Student and one Examination.

Relationships: a Student registers for many Courses and a Course has many Students (many-to-many through Registration); a Faculty teaches one or more Courses; ExaminationStaff conducts Examinations for Courses; and each Result is linked to exactly one Student and one Examination.

### 2.3 Object-Oriented Principles Applied

- Encapsulation: The Result class keeps raw marks private and exposes only a computed getGrade() method, preventing external classes from tampering with evaluation data directly.
- Inheritance: Student, Faculty, ExaminationStaff, and Administrator all inherit common identity attributes and login behaviour from the Person base class, avoiding duplicated code.
- Polymorphism: A method such as viewDashboard() is overridden per subclass – a Student sees registered courses and results, a Faculty member sees classes and grading tasks, and an Administrator sees system-wide reports – through the same method signature.

### 2.4 Recommended SDLC Model

The Incremental Model is recommended. The four core modules – admissions, registration, examination, and results – are relatively independent and can be built, tested, and delivered one at a time. This allows the university to start using the admissions and registration modules while the examination and result modules are still being developed, get early feedback from faculty and administrative staff, and reduce integration risk compared to a single, all-at-once Waterfall delivery. Because the overall requirements of a university's academic calendar are well understood in advance, a purely exploratory Agile approach is not strictly necessary, but the incremental, module-by-module delivery still provides the flexibility to refine each module before moving to the next.

---

## 3. Online Retail Shopping Platform

*A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory. Question: Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.*

**(10 Marks)**

**ANSWER**

### 3.1 Achieving Modular Design through Object-Oriented Techniques

Modularity is achieved by decomposing the platform into independent, self-contained classes that each own a single responsibility and communicate through well-defined interfaces rather than shared internal data. For the retail platform, natural modules include Product, Cart, Order, Payment, Delivery, Vendor, and Inventory. Because each class encapsulates its own attributes and operations, a change to how payments are processed, for instance, does not require any change to the Cart or Delivery classes. Classes are further organised through inheritance (a User base class with Customer, Vendor, and Administrator subclasses) so that common behaviour is written once and reused, and through polymorphism (for example, a trackStatus() method that behaves differently for an Order object versus a Delivery object) so that new order or delivery types can be added without modifying existing calling code. This combination of

encapsulation, inheritance, and polymorphism is what allows the platform to be built, tested, and maintained as loosely coupled, independently replaceable modules.

### 3.2 Evaluation of SDLC Models

- Waterfall: Unsuitable here because e-commerce requirements (new payment options, promotional features, recommendation logic) change frequently after launch, and Waterfall does not accommodate requirement changes once a phase is complete.
- Spiral Model: Offers strong risk management, which is useful given the financial and security risk in online payments, but its heavy emphasis on risk analysis in every cycle makes it slower and more process-intensive than the platform's time-to-market needs justify.
- Agile (Scrum): Well suited because it delivers working modules (catalog browsing, cart, checkout, payment, delivery tracking) in short sprints, allows the vendor-facing catalog features and customer-facing shopping features to evolve independently, and lets the company respond quickly to market feedback and seasonal demand.

### 3.3 Recommended SDLC Model

The Agile (Scrum) model is the most appropriate choice for this scenario. Its iterative sprint cycles align naturally with the platform's modular, object-oriented class structure – each sprint can focus on delivering or refining one module (for example, the Order and Payment classes in one sprint, and the Delivery-tracking classes in the next) – while continuous stakeholder feedback ensures that vendor cataloging tools and customer shopping features stay aligned with real business needs as the platform grows.

---

## 4. Smart Waste Management System

*A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application. Question: Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.*

**(10 Marks)**

### ANSWER

### 4.1 System Understanding

The Smart Waste Management System integrates physical IoT hardware (sensors embedded in bins) with software that processes real-time fill-level data, notifies collection vehicles, and lets citizens report issues through a mobile app. This mix of hardware sensing, real-time data processing, and a citizen-facing application makes object-oriented design especially valuable, since each concern can be modelled as a separate, well-encapsulated class.

### 4.2 Object-Oriented Design

- Bin: binId, location, capacity, currentLevel | isFull(), sendAlert().
- Sensor: sensorId, type, lastReading | captureData() – associated with one Bin.

- CollectionVehicle: vehicleId, route, capacity | receiveNotification(), updateRoute().
- Citizen: citizenId, name, contactNumber | reportIssue().
- ComplaintReport: reportId, description, status, dateReported | trackStatus().
- Municipality/Administrator: adminId | monitorBins(), manageVehicles(), viewReports().

### 4.3 Object-Oriented Principles Applied

- Encapsulation: The Bin class hides the raw sensor threshold logic behind an isFull() method, so the CollectionVehicle class only needs to know the result (full or not full), not how fill level is calculated.
- Inheritance: A common Device base class can be extended by Sensor and any future hardware types (e.g., a lid-status sensor), while a User base class is extended by Citizen and Administrator to share login and profile behaviour.
- Polymorphism: A notify() method is overridden differently depending on the recipient – sending a route update to a CollectionVehicle versus an app notification to a Citizen – through the same method call defined at a common level.

### 4.4 Recommended Life Cycle Model

The Spiral Model is best suited for this system. Smart waste management combines real-time sensor data, hardware-software integration, and requirements that are likely to evolve as the municipality learns from live deployment (for example, adjusting fill-level thresholds, adding new sensor types, or integrating with additional city systems). The Spiral Model's repeated cycles of planning, risk analysis, prototyping, and evaluation make it possible to prototype the IoT integration early, identify hardware-communication risks (sensor connectivity, data latency) before committing to a full build, and progressively refine both the sensor-processing logic and the citizen-reporting features as real-world usage reveals new requirements. This risk-driven, iterative structure is better matched to the uncertainty of IoT hardware integration than a purely sequential model like Waterfall or a pure feature-sprint model like Scrum, though incremental delivery within each spiral cycle can still be used to release usable functionality early.

**RUBRICS FOR CALCULATION:**

# Case study

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Case | 25 | Thorough understanding; clearly identifies all key issues | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly |
| Application of Concepts | 25 | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps | Limited application of concepts | Fails to apply relevant concepts |
| Analysis | 20 | In-depth analysis with strong reasoning and insights | Good analysis with logical reasoning | Basic analysis with limited depth | Weak or no analysis; lacks reasoning |
| Solution Design | 20 | Innovative, feasible solutions with strong justification | Logical solutions with adequate justification | Basic solutions with weak justification | Incorrect or no solution provided |
| Clarity | 10 | Clear, well-structured, and easy to understand | Organized and understandable presentation | Limited clarity and structure | Poor clarity; difficult to understand |